Relazione – Pratica S2/L4: Python & AI

Introduzione

Durante questa esercitazione è stato configurato un ambiente Python per l'utilizzo delle API Gemini, con l'obiettivo di sviluppare semplici applicazioni di intelligenza artificiale applicate alla Cyber Security.

Obiettivo

L'obiettivo della pratica è comprendere i fondamenti dell'interazione con un modello linguistico (LLM) tramite Python, realizzando uno script base e un chatbot interattivo.

Strumenti utilizzati

- Sistema operativo: Kali Linux
- Linguaggio: Python
- Librerie: google-genai, pandas, numpy
- Virtual Environment (venv)
- API Gemini (Google AI Studio)

Attività svolte

1. Configurazione ambiente

È stato creato un ambiente virtuale Python (**venv_ai**) e sono state installate le librerie necessarie.

```
┌──(kali㉿kali)-[~]
└─$ python -m venv venv_ai

┌──(kali㉿kali)-[~]
└─$ source venv_ai/bin/activate

┌──(venv_ai)─(kali㉿kali)-[~]
└─$ pip install google-genai pandas numpy
```

Successivamente è stata configurata la variabile d'ambiente **GEMINI_API_KEY** per permettere l'autenticazione alle API.

```
┌──(venv_ai)─(kali㉿kali)-[~]
└─$ echo 'export GEMINI_API_KEY="AQ.Ab8RN6IQLbUsM ... OsNYL7GsROA"' >> ~/.zshrc
```

2. Script Hello Gemini

È stato creato un primo script (**hello_gemini.py**) per inviare un prompt al modello AI e stampare la risposta.

```
┌──(venv_ai)─(kali㉿kali)-[~]
└─$ nano hello_gemini.py
```

```
  GNU nano 8.7.1                                    hello_gemini.py *
from google import genai
import os

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

response = client.models.generate_content(
    model="gemini-flash-latest",
    contents="Ciao! Chi sei?"
)

print(response.text)
```

Questo passaggio è servito per verificare il corretto funzionamento delle API e dell'ambiente configurato.

3. Chatbot Security Assistant

È stato sviluppato un chatbot interattivo (**chatbot.py**) che utilizza:

- un system prompt per definire il comportamento dell'AI come tutor di cybersecurity
- un ciclo **while** per mantenere attiva la conversazione
- un comando "**esci**" per terminare il programma

```
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                                   chatbot.py
from google import genai
from google.genai import types
import os

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

config = types.GenerateContentConfig(
    system_instruction="""
    Sei un tutor di sicurezza informatica per studenti principianti.
    Spiega in modo semplice e chiaro.
    """
)

while True:
    domanda = input("Tu: ")

    if domanda.lower() == "esci":
        print("Arrivederci!")
        break

    response = client.models.generate_content(
        model="gemini-flash-latest",
        contents=domanda,
        config=config
    )

    print("AI:", response.text)
```

Il chatbot è stato utilizzato per rispondere a domande di cybersecurity (*es. firewall, brute force, SQL injection*), dimostrando il funzionamento del ciclo di conversazione.

Risultati ottenuti

- Lo script Hello Gemini ha restituito correttamente risposte generate dall'AI
- Il chatbot ha funzionato correttamente in modalità interattiva
- Le risposte sono risultate coerenti, chiare e adatte a un contesto didattico

```
┌──(venv_ai)─(kali㉿kali)-[~]
└─$ python chatbot.py
Tu: cos'è un firewall
AI: Ciao! Come tuo tutor di sicurezza informatica, sono felice di spiegarti questo concetto fondamentale.

Per capire cos'è un **firewall**, proviamo a usare un'analogia semplice.

### L'analogia del "Buttafuori"
```

```
Tu: cos'è brute force
AI: Ciao! Come tuo tutor di sicurezza informatica, sono felice di spiegartelo in modo semplice.

### Cos'è il Brute Force?

Immagina di aver dimenticato la combinazione del lucchetto della tua bicicletta. È un lucchetto a 3 cifre (da 000 a
 999). Cosa fai per aprirlo?
```

```
Tu: cos'è SQL injection
AI: Ciao! Benvenuto in questa lezione. Oggi parleremo di una delle vulnerabilità più famose e pericolose del web: l
a **SQL Injection** (spesso abbreviata in **SQLi**).

Immagina di essere un investigatore che cerca di capire come un ladro possa entrare in una cassaforte non forzando
la serratura, ma "ingannando" chi ha le chiavi.
```

Conclusione

L'esercitazione ha permesso di comprendere come configurare un ambiente Python e utilizzare un modello AI tramite API.

È stato possibile sviluppare sia uno script base sia un chatbot interattivo, dimostrando la comprensione dei concetti fondamentali.